PDR.cloud Technical Assessment Challenge

Objective

In this challenge, you will demonstrate your fullstack development skills using Angular, Angular Material, NestJS and a shared monorepo architecture. The task is structured to assess your ability to architect frontend and backend solutions in a modular, scalable and maintainable way.

Technologies

- **Monorepo**: Nx (preferred), NPM Workspaces or Turborepo
- **Frontend**: Angular 18+, Angular Material, SCSS
- **Backend**: NestJS
- **Validation**: Zod (preferred) or class-validator
- **Language**: TypeScript

Project Structure

Set up a **monorepo** with the following structure:

```
apps/
  frontend/    → Angular app
  api/         → NestJS backend
libs/
  shared/      → DTOs, validation schemas, interfaces
```

Use Nx or another workspace tool to manage and run both apps from a single codebase.

Part 1: Backend API with NestJS

Set up a NestJS API that exposes endpoints and stores user data persistently.

Endpoints

Use the provided `users.json` file with 100 users as initial data

- `GET /users`
  → Returns a list of users
- `POST /users`
  → Accepts a new user and persists it to a JSON file (`data/users.json` or similar)
- `GET /users/:id`
  → Returns a single user by ID

Requirements

- Use `Zod` (preferred) or `class-validator` for input validation.
- Create a **shared validation schema** in `libs/shared` and use it in both backend and frontend.
- The user model includes:
  - `id` (number)
  - `firstName` (string)
  - `lastName` (string)
  - `email` (string)
  - `phoneNumber` (string)
  - `birthDate` (ISO date string)
  - `role`: `admin`, `editor`, or `viewer` (enum)

Persistence

- Store users in-memory and persist them to a file (`data/users.json`).
- Ensure consistent read/write behavior (e.g., use a queue or simple locking).

Part 2: Angular Frontend with Angular Material

Theme

Create a custom Angular Material **Material 3** theme using the following colors:

- `primary`: `#1da4e8`
- `secondary`: `#20d2a8`
- `tertiary`: `#e4dc46`
- `error`: `#d32f55`

Use SCSS or the Material Theme API to apply the palette globally.

Components

User List

- Display user data in a **Material table**
- Each row of the table should show a user's id, name, email and role.
- Enable **pagination (25 users per page)**
- Support **searching by full name**
- Fetch users from the `GET /users` endpoint

User Details Dialog

- Clicking a user row opens a Material Dialog
- Load user data using `GET /users/:id`
- Display: full name, email, phone, birth date, and role

User Creation Form

- Use Angular **Reactive Forms**
- Implement validation using the shared Zod schema
- Submit to `POST /users` and reload the user list
- Display success and error messages using Angular Material Snackbar

Mandatory Complexity: Conditional Validation Based on Role

Challenge:

Implement a dynamic validation schema that changes depending on the selected `role` in the creation form.

Rules

- `admin`: requires `phone` **and** `birthDate`
- `editor`: requires `phone`
- `viewer`: no additional required fields

Requirements

- The dynamic schema must be implemented using **Zod refinement/superRefine**
- The validation logic should **synchronize across frontend and backend**
- The UI should display errors appropriately per role
- Validation schema is stored in `libs/shared` and imported in both projects

Hint

- Use `zod.discriminatedUnion` or `.refine()` or `.superRefine()` to enforce conditional logic.
- This is a non-trivial task requiring deeper understanding of form state, live validation updates, and schema-sharing.

Part 3: UI Component Challenge – Pure CSS Smiley (Optional)

Create a standalone UI component using only HTML and (S)CSS — no images or SVGs. This component should be a responsive smiley face, styled purely with CSS layout techniques.

This task will test your ability to work with modern layout tools like Flexbox and CSS Grid, as well as your understanding of responsive design without relying on positioning hacks or external assets.

Requirements

- The smiley must be responsive and scale based on the viewport.
- Use Flexbox or CSS Grid to position the eyes and mouth.
- Do not use position: absolute, images, or SVGs.
- (Optional but encouraged): Use pseudo-elements (::before, ::after) to add flair or complexity.

Integration

- The smiley should be implemented as a standalone Angular component called SmileyComponent.
- Add a new Angular route /smiley that loads this component in the frontend.
- The component should be part of the Angular Material theme and fit the overall styling of the app.

Outcome

- Fully working Angular route at /smiley
- One Angular component (SmileyComponent) with embedded SCSS
- No external assets
- Responsive design
- Clean and maintainable layout using modern CSS

Submission Guidelines

- Push your solution to a public GitHub repository.
- The repository should include:
    - A README.md with clear instructions on how to:
        - Install dependencies
        - Start the backend and frontend apps (e.g., nx serve api, nx serve frontend)
        - Run any scripts for data (if needed)
    - Notes on:
        - Assumptions or architectural decisions
        - Known limitations (if any)
    - Ensure that:
        - The code is cleanly structured, follows best practices, and is well-documented.
        - The libs/shared folder contains shared types and validation logic used in both Angular and NestJS.
        - The /smiley route works in the Angular app and showcases the responsive CSS-only design.